```
import React, { useState, useEffect, useRef } from 'react';
import {
  TrendingUp,
  Filter,
  Search,
  AlertCircle,
  CheckCircle,
  Download,
  Bell,
  Settings,
  BarChart3,
  Zap,
  Clock,
  Volume2,
  Target,
  Eye,
  Copy,
  X,
} from 'lucide-react';

export default function AdvancedPreBreakoutScanner() {
  const [stocks, setStocks] = useState([]);
  const [filtered, setFiltered] = useState([]);
  const [searchTerm, setSearchTerm] = useState('');
  const [signalFilter, setSignalFilter] = useState('All');
  const [qualityFilter, setQualityFilter] = useState('All');
  const [selectedStock, setSelectedStock] = useState(null);
  const [sortBy, setSortBy] = useState('score');
  const [alerts, setAlerts] = useState([]);
  const [showAlerts, setShowAlerts] = useState(false);
  const [watchlist, setWatchlist] = useState([]);
  const [apiConfig, setApiConfig] = useState({
    kiteApiKey: '',
    kiteAccessToken: '',
    useLiveData: false,
  });
  const [showSettings, setShowSettings] = useState(false);
  const [lastUpdate, setLastUpdate] = useState(null);
  const [autoRefresh, setAutoRefresh] = useState(true);
  const [refreshInterval, setRefreshInterval] = useState(60);

  // Generate mock stock data
  const generateMockStocks = () => {
    const mockStocks = [
```

```
{
  id: 1,
  symbol: 'RELIANCE',
  score: 92,
  signal: 'Very Strong',
  fromPivot: 2.3,
  stage: 2,
  rsRank: 85,
  baseQuality: 88,
  volQuality: 82,
  vcp: true,
  volDry: true,
  rsi: 58,
  adx: 42,
  risk: 'Low',
  accuracy: 92,
  currentPrice: 2850,
  pivot: 2785,
  support1: 2720,
  support2: 2650,
  resistance1: 2920,
  resistance2: 3050,
  avgVolume: 15420000,
  volume52week: 16850000,
  marketCap: '2.8T',
  sector: 'Energy',
  trend: 'Bullish',
  breakoutProbability: 0.78,
},
{
  id: 2,
  symbol: 'INFY',
  score: 88,
  signal: 'Very Strong',
  fromPivot: 1.8,
  stage: 2,
  rsRank: 82,
  baseQuality: 85,
  volQuality: 79,
  vcp: true,
  volDry: true,
  rsi: 62,
  adx: 39,
  risk: 'Low',
  accuracy: 88,
  currentPrice: 1920,
  pivot: 1887,
```

```
  support1: 1850,
  support2: 1800,
  resistance1: 1980,
  resistance2: 2050,
  avgVolume: 8320000,
  volume52week: 9100000,
  marketCap: '8.5T',
  sector: 'IT',
  trend: 'Bullish',
  breakoutProbability: 0.75,
},
{
  id: 3,
  symbol: 'TCS',
  score: 84,
  signal: 'Strong',
  fromPivot: 3.1,
  stage: 2,
  rsRank: 78,
  baseQuality: 82,
  volQuality: 75,
  vcp: true,
  volDry: false,
  rsi: 56,
  adx: 36,
  risk: 'Low',
  accuracy: 84,
  currentPrice: 3450,
  pivot: 3345,
  support1: 3280,
  support2: 3200,
  resistance1: 3520,
  resistance2: 3620,
  avgVolume: 5210000,
  volume52week: 5800000,
  marketCap: '12.3T',
  sector: 'IT',
  trend: 'Bullish',
  breakoutProbability: 0.72,
},
{
  id: 4,
  symbol: 'HCLTECH',
  score: 79,
  signal: 'Strong',
  fromPivot: 2.7,
  stage: 2,
```

```
  rsRank: 75,
  baseQuality: 78,
  volQuality: 72,
  vcp: false,
  volDry: true,
  rsi: 54,
  adx: 33,
  risk: 'Medium',
  accuracy: 79,
  currentPrice: 1685,
  pivot: 1640,
  support1: 1610,
  support2: 1570,
  resistance1: 1730,
  resistance2: 1800,
  avgVolume: 3450000,
  volume52week: 3900000,
  marketCap: '4.2T',
  sector: 'IT',
  trend: 'Bullish',
  breakoutProbability: 0.68,
},
{
  id: 5,
  symbol: 'WIPRO',
  score: 75,
  signal: 'Strong',
  fromPivot: 4.2,
  stage: 2,
  rsRank: 71,
  baseQuality: 74,
  volQuality: 68,
  vcp: true,
  volDry: true,
  rsi: 52,
  adx: 30,
  risk: 'Medium',
  accuracy: 75,
  currentPrice: 498,
  pivot: 478,
  support1: 465,
  support2: 450,
  resistance1: 515,
  resistance2: 535,
  avgVolume: 12450000,
  volume52week: 13500000,
  marketCap: '2.1T',
```

```
  sector: 'IT',
  trend: 'Neutral',
  breakoutProbability: 0.65,
},
{
  id: 6,
  symbol: 'BAJAJFINSV',
  score: 82,
  signal: 'Very Strong',
  fromPivot: 1.5,
  stage: 2,
  rsRank: 80,
  baseQuality: 86,
  volQuality: 81,
  vcp: true,
  volDry: true,
  rsi: 60,
  adx: 40,
  risk: 'Low',
  accuracy: 86,
  currentPrice: 1785,
  pivot: 1759,
  support1: 1720,
  support2: 1680,
  resistance1: 1850,
  resistance2: 1920,
  avgVolume: 6540000,
  volume52week: 7100000,
  marketCap: '1.8T',
  sector: 'Financials',
  trend: 'Bullish',
  breakoutProbability: 0.76,
},
{
  id: 7,
  symbol: 'MARUTI',
  score: 73,
  signal: 'Strong',
  fromPivot: 3.8,
  stage: 2,
  rsRank: 68,
  baseQuality: 71,
  volQuality: 65,
  vcp: false,
  volDry: false,
  rsi: 50,
  adx: 28,
```

```
      risk: 'High',
      accuracy: 70,
      currentPrice: 12450,
      pivot: 12000,
      support1: 11850,
      support2: 11650,
      resistance1: 12600,
      resistance2: 12850,
      avgVolume: 2340000,
      volume52week: 2850000,
      marketCap: '1.2T',
      sector: 'Auto',
      trend: 'Neutral',
      breakoutProbability: 0.62,
    },
    {
      id: 8,
      symbol: 'BHARTIARTL',
      score: 80,
      signal: 'Very Strong',
      fromPivot: 2.1,
      stage: 2,
      rsRank: 79,
      baseQuality: 84,
      volQuality: 80,
      vcp: true,
      volDry: true,
      rsi: 61,
      adx: 41,
      risk: 'Low',
      accuracy: 85,
      currentPrice: 1550,
      pivot: 1520,
      support1: 1490,
      support2: 1450,
      resistance1: 1620,
      resistance2: 1680,
      avgVolume: 9870000,
      volume52week: 10500000,
      marketCap: '5.5T',
      sector: 'Telecom',
      trend: 'Bullish',
      breakoutProbability: 0.74,
    },
  ];
  return mockStocks;
};
```

```
// Initialize data
useEffect(() => {
  setStocks(generateMockStocks());
  setLastUpdate(new Date());
}, []);

// Simulate real-time data refresh
useEffect(() => {
  if (!autoRefresh) return;

  const interval = setInterval(() => {
    // Simulate price updates
    setStocks((prevStocks) =>
      prevStocks.map((stock) => ({
        ...stock,
        currentPrice: stock.currentPrice * (1 + (Math.random() - 0.5) * 0.005),
        rsi: Math.max(20, Math.min(80, stock.rsi + (Math.random() - 0.5) * 2)),
        adx: Math.max(10, Math.min(70, stock.adx + (Math.random() - 0.5) * 1)),
      }))
    );
    setLastUpdate(new Date());
  }, refreshInterval * 1000);

  return () => clearInterval(interval);
}, [autoRefresh, refreshInterval]);

// Apply filters
useEffect(() => {
  let result = [...stocks];

  if (signalFilter !== 'All') {
    result = result.filter((s) => s.signal === signalFilter);
  }

  if (qualityFilter === 'HighBase') {
    result = result.filter((s) => s.baseQuality >= 80);
  } else if (qualityFilter === 'VolConfirmed') {
    result = result.filter((s) => s.volDry);
  } else if (qualityFilter === 'LeadingRS') {
    result = result.filter((s) => s.rsRank >= 75);
  } else if (qualityFilter === 'LowRisk') {
    result = result.filter((s) => s.risk === 'Low');
  } else if (qualityFilter === 'HighAccuracy') {
    result = result.filter((s) => s.accuracy >= 85);
  }
```

```
  if (searchTerm) {
    result = result.filter((s) =>
      s.symbol.toLowerCase().includes(searchTerm.toLowerCase())
    );
  }

  result.sort((a, b) => {
    if (sortBy === 'score') return b.score - a.score;
    if (sortBy === 'rsRank') return b.rsRank - a.rsRank;
    if (sortBy === 'fromPivot') return a.fromPivot - b.fromPivot;
    if (sortBy === 'breakoutProbability')
      return b.breakoutProbability - a.breakoutProbability;
    return 0;
  });

  setFiltered(result);
}, [stocks, searchTerm, signalFilter, qualityFilter, sortBy]);

// Add to watchlist
const addToWatchlist = (stock) => {
  if (!watchlist.find((s) => s.id === stock.id)) {
    setWatchlist([...watchlist, stock]);
    addAlert(`${stock.symbol} added to watchlist`, 'success');
  }
};

// Create price alert
const createAlert = (stock, alertType = 'breakout') => {
  const alertMessage =
    alertType === 'breakout'
      ? `Alert set: ${stock.symbol} breaks above ₹${stock.pivot}`
      : `Alert set: ${stock.symbol} bounces from ₹${stock.support1}`;

  addAlert(alertMessage, 'info');
};

// Add alert to list
const addAlert = (message, type = 'info') => {
  const newAlert = {
    id: Date.now(),
    message,
    type,
    timestamp: new Date(),
  };
  setAlerts((prev) => [newAlert, ...prev].slice(0, 10));
};
```

```
// Export to CSV
const exportToCSV = () => {
  if (filtered.length === 0) {
    addAlert('No data to export', 'warning');
    return;
  }

  const headers = [
    'Symbol',
    'Score',
    'Signal',
    'From Pivot %',
    'Stage',
    'RS Rank',
    'Base Quality',
    'Vol Quality',
    'VCP',
    'Vol Dry',
    'Risk',
    'Accuracy',
    'Current Price',
    'Pivot',
    'Support 1',
    'Resistance 1',
    'RSI',
    'ADX',
    'Sector',
    'Trend',
    'Breakout Probability',
  ];

  const rows = filtered.map((stock) => [
    stock.symbol,
    stock.score,
    stock.signal,
    stock.fromPivot.toFixed(2),
    stock.stage,
    stock.rsRank,
    stock.baseQuality,
    stock.volQuality,
    stock.vcp ? 'Yes' : 'No',
    stock.volDry ? 'Yes' : 'No',
    stock.risk,
    stock.accuracy,
    stock.currentPrice.toFixed(2),
    stock.pivot.toFixed(2),
    stock.support1.toFixed(2),
```

```
    stock.resistance1.toFixed(2),
    stock.rsi.toFixed(2),
    stock.adx.toFixed(2),
    stock.sector,
    stock.trend,
    (stock.breakoutProbability * 100).toFixed(1),
  ]);

  const csv = [
    headers.join(','),
    ...rows.map((row) =>
      row
        .map((cell) => (typeof cell === 'string' ? `"${cell}"` : cell))
        .join(',')
    ),
  ].join('\n');

  const blob = new Blob([csv], { type: 'text/csv' });
  const url = window.URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = `PreBreakout_Scanner_${new Date().toISOString().split('T')[0]}.c
  a.click();
  window.URL.revokeObjectURL(url);

  addAlert(`✓ Exported ${filtered.length} stocks to CSV`, 'success');
};

// Get TradingView widget code
const getTradingViewWidget = (symbol) => {
  return `<div class="tradingview-widget-container">
    <div id="tradingview_${symbol}"></div>
    <script type="text/javascript" src="https://s3.tradingview.com/tv.js"></scr
    <script type="text/javascript">
      new TradingView.widget({
        "autosize": true,
        "symbol": "NSE:${symbol}",
        "interval": "D",
        "timezone": "Asia/Kolkata",
        "theme": "dark",
        "style": "1",
        "locale": "in",
        "toolbar_bg": "#f1f3f6",
        "enable_publishing": false,
        "allow_symbol_change": true,
        "container_id": "tradingview_${symbol}"
      });
```

```
    </script>
  </div>`;
};

// Copy to clipboard
const copyToClipboard = (text) => {
  navigator.clipboard.writeText(text);
  addAlert('Copied to clipboard!', 'success');
};

const getSignalColor = (signal) => {
  if (signal === 'Very Strong')
    return 'bg-gradient-to-r from-green-500/20 to-emerald-500/20 text-green-400
  if (signal === 'Strong')
    return 'bg-gradient-to-r from-blue-500/20 to-cyan-500/20 text-blue-400 bord
  return 'bg-gray-500/20 text-gray-400 border border-gray-500/30';
};

const getRiskColor = (risk) => {
  if (risk === 'Low') return 'text-green-400';
  if (risk === 'Medium') return 'text-yellow-400';
  return 'text-red-400';
};

const getTrendColor = (trend) => {
  if (trend === 'Bullish') return 'text-green-400';
  if (trend === 'Bearish') return 'text-red-400';
  return 'text-yellow-400';
};

return (
  <div
    className="min-h-screen text-gray-100"
    style={{
      background:
        'linear-gradient(135deg, #0f172a 0%, #1a1f3a 50%, #1e293b 100%)',
    }}
  >
    {/* Header */}
    <div className="border-b border-slate-700/50 backdrop-blur-lg sticky top-0
      <div className="max-w-7xl mx-auto px-4 py-5">
        <div className="flex items-center justify-between mb-4">
          <div className="flex items-center gap-4">
            <div className="p-3 bg-gradient-to-br from-emerald-500 via-teal-600
              <TrendingUp className="w-7 h-7 text-white" />
            </div>
            <div>
```

```
        <h1 className="text-3xl font-bold bg-gradient-to-r from-emerald-4
          Pre-Breakout Scanner Pro
        </h1>
        <p className="text-xs text-slate-400 mt-1">
          VCP & Stage 2 Strategy • Real-time NSE Screening
        </p>
      </div>
    </div>
    <div className="flex items-center gap-2">
      <button
        onClick={() => setShowAlerts(!showAlerts)}
        className="relative p-2 hover:bg-slate-700/50 rounded-lg transiti
      >
        <Bell className="w-5 h-5" />
        {alerts.length > 0 && (
          <span className="absolute top-1 right-1 w-2 h-2 bg-red-500 roun
        )}
      </button>
      <button
        onClick={() => setShowSettings(!showSettings)}
        className="p-2 hover:bg-slate-700/50 rounded-lg transition-colors
      >
        <Settings className="w-5 h-5" />
      </button>
    </div>
  </div>

  {/* Status Bar */}
  <div className="flex items-center justify-between text-xs">
    <div className="flex items-center gap-4">
      <div className="flex items-center gap-1 text-slate-400">
        <div className="w-2 h-2 bg-green-500 rounded-full animate-pulse"
        Last Update: {lastUpdate?.toLocaleTimeString()}
      </div>
      <div className="text-slate-400">
        Stocks Found: <span className="text-emerald-400 font-semibold">{f
      </div>
      <div className="text-slate-400">
        Watchlist: <span className="text-emerald-400 font-semibold">{watc
      </div>
    </div>
    <button
      onClick={exportToCSV}
      className="flex items-center gap-2 px-3 py-1 bg-emerald-600/20 hove
    >
      <Download className="w-4 h-4" />
      Export CSV
```

```
      </button>
    </div>
  </div>
</div>

{/* Alerts Panel */}
{showAlerts && (
  <div className="fixed top-20 right-4 w-80 bg-slate-900 border border-slat
    <div className="sticky top-0 bg-slate-800 px-4 py-3 border-b border-sla
      <h3 className="font-semibold text-white text-sm">Alerts ({alerts.leng
      <button
        onClick={() => setShowAlerts(false)}
        className="text-slate-400 hover:text-white"
      >
        <X className="w-4 h-4" />
      </button>
    </div>
    <div className="p-4 space-y-2">
      {alerts.length === 0 ? (
        <p className="text-xs text-slate-500">No alerts yet</p>
      ) : (
        alerts.map((alert) => (
          <div
            key={alert.id}
            className={`p-3 rounded-lg text-xs border ${
              alert.type === 'success'
                ? 'bg-green-500/10 border-green-500/30 text-green-400'
                : alert.type === 'warning'
                ? 'bg-yellow-500/10 border-yellow-500/30 text-yellow-400'
                : 'bg-blue-500/10 border-blue-500/30 text-blue-400'
            }`}
          >
            <div className="flex justify-between items-start">
              <span className="flex-1">{alert.message}</span>
              <span className="text-xs opacity-70 ml-2">
                {alert.timestamp.toLocaleTimeString()}
              </span>
            </div>
          </div>
        ))
      )}
    </div>
  </div>
)}

{/* Settings Panel */}
{showSettings && (
```

```
<div className="fixed inset-0 bg-black/60 backdrop-blur-sm z-50 flex item
  <div className="bg-slate-900 border border-slate-700 rounded-xl max-w-m
    <div className="flex justify-between items-center mb-4">
      <h2 className="text-lg font-bold text-white">Settings & API Config<
      <button
        onClick={() => setShowSettings(false)}
        className="text-slate-400 hover:text-white"
      >
        <X className="w-5 h-5" />
      </button>
    </div>

    <div className="space-y-4">
      {/* Auto Refresh */}
      <div>
        <label className="flex items-center gap-2 mb-2">
          <input
            type="checkbox"
            checked={autoRefresh}
            onChange={(e) => setAutoRefresh(e.target.checked)}
            className="w-4 h-4"
          />
          <span className="text-sm text-white">Auto-refresh data</span>
        </label>
        {autoRefresh && (
          <div>
            <label className="block text-xs text-slate-400 mb-1">
              Refresh interval (seconds)
            </label>
            <input
              type="number"
              value={refreshInterval}
              onChange={(e) => setRefreshInterval(Math.max(10, parseInt(e
              min="10"
              max="300"
              className="w-full px-3 py-2 bg-slate-700/50 border border-s
            />
          </div>
        )}
      </div>

      {/* Kite API Config */}
      <div className="border-t border-slate-700 pt-4">
        <h3 className="text-sm font-semibold text-white mb-3">Zerodha Kit
        <div>
          <label className="block text-xs text-slate-400 mb-1">API Key</l
          <input
```

```
        type="password"
        placeholder="Your Kite API Key"
        value={apiConfig.kiteApiKey}
        onChange={(e) =>
          setApiConfig({
            ...apiConfig,
            kiteApiKey: e.target.value,
          })
        }
        className="w-full px-3 py-2 bg-slate-700/50 border border-sla
      />
    </div>
    <div>
      <label className="block text-xs text-slate-400 mb-1">Access Tok
      <input
        type="password"
        placeholder="Your Access Token"
        value={apiConfig.kiteAccessToken}
        onChange={(e) =>
          setApiConfig({
            ...apiConfig,
            kiteAccessToken: e.target.value,
          })
        }
        className="w-full px-3 py-2 bg-slate-700/50 border border-sla
      />
    </div>
    <label className="flex items-center gap-2">
      <input
        type="checkbox"
        checked={apiConfig.useLiveData}
        onChange={(e) =>
          setApiConfig({
            ...apiConfig,
            useLiveData: e.target.checked,
          })
        }
        className="w-4 h-4"
      />
      <span className="text-sm text-white">Use live Kite data</span>
    </label>
  </div>

  <button
    onClick={() => {
      if (apiConfig.kiteApiKey && apiConfig.kiteAccessToken) {
        addAlert('API configured successfully!', 'success');
```

```
              setShowSettings(false);
            } else {
              addAlert('Please fill in all API fields', 'warning');
            }
          }}
          className="w-full mt-4 px-4 py-2 bg-gradient-to-r from-emerald-60
        >
          Save Configuration
        </button>
      </div>
    </div>
  </div>
)}

{/* Main Content */}
<div className="max-w-7xl mx-auto px-4 py-6">
  {/* Filters */}
  <div className="bg-slate-800/40 backdrop-blur border border-slate-700/50
    <div className="grid grid-cols-1 md:grid-cols-5 gap-4">
      <div>
        <label className="block text-xs font-semibold text-slate-300 mb-2">
          SEARCH
        </label>
        <div className="relative">
          <Search className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h
          <input
            type="text"
            placeholder="e.g., RELIANCE"
            value={searchTerm}
            onChange={(e) => setSearchTerm(e.target.value)}
            className="w-full pl-9 pr-3 py-2 bg-slate-700/50 border border-
          />
        </div>
      </div>

      <div>
        <label className="block text-xs font-semibold text-slate-300 mb-2">
          SIGNAL
        </label>
        <select
          value={signalFilter}
          onChange={(e) => setSignalFilter(e.target.value)}
          className="w-full px-3 py-2 bg-slate-700/50 border border-slate-6
        >
          <option>All</option>
          <option>Very Strong</option>
          <option>Strong</option>
```

```
  </select>
</div>

<div>
  <label className="block text-xs font-semibold text-slate-300 mb-2">
    QUALITY
  </label>
  <select
    value={qualityFilter}
    onChange={(e) => setQualityFilter(e.target.value)}
    className="w-full px-3 py-2 bg-slate-700/50 border border-slate-6
  >
    <option value="All">All</option>
    <option value="HighBase">High Base</option>
    <option value="VolConfirmed">Vol Confirmed</option>
    <option value="LeadingRS">Leading RS</option>
    <option value="LowRisk">Low Risk</option>
    <option value="HighAccuracy">High Accuracy</option>
  </select>
</div>

<div>
  <label className="block text-xs font-semibold text-slate-300 mb-2">
    SORT BY
  </label>
  <select
    value={sortBy}
    onChange={(e) => setSortBy(e.target.value)}
    className="w-full px-3 py-2 bg-slate-700/50 border border-slate-6
  >
    <option value="score">Score</option>
    <option value="rsRank">RS Rank</option>
    <option value="fromPivot">From Pivot</option>
    <option value="breakoutProbability">Breakout Probability</option>
  </select>
</div>

<div>
  <label className="block text-xs font-semibold text-slate-300 mb-2">
     
  </label>
  <button
    onClick={() => {
      setSearchTerm('');
      setSignalFilter('All');
      setQualityFilter('All');
      setSortBy('score');
```

```
        }}
        className="w-full px-4 py-2 bg-slate-700/50 hover:bg-slate-700 bo
      >
        Reset Filters
      </button>
    </div>
  </div>
</div>

{/* Table */}
{filtered.length > 0 ? (
  <div className="bg-slate-800/40 backdrop-blur border border-slate-700/5
    <div className="overflow-x-auto">
      <table className="w-full text-sm">
        <thead>
          <tr className="bg-slate-900/50 border-b border-slate-700/50">
            <th className="px-4 py-3 text-left font-semibold text-slate-3
              SYMBOL
            </th>
            <th className="px-4 py-3 text-center font-semibold text-slate
              SCORE
            </th>
            <th className="px-4 py-3 text-center font-semibold text-slate
              SIGNAL
            </th>
            <th className="px-4 py-3 text-center font-semibold text-slate
              FROM PIVOT
            </th>
            <th className="px-4 py-3 text-center font-semibold text-slate
              RS
            </th>
            <th className="px-4 py-3 text-center font-semibold text-slate
              BASE
            </th>
            <th className="px-4 py-3 text-center font-semibold text-slate
              VCP
            </th>
            <th className="px-4 py-3 text-center font-semibold text-slate
              BREAKOUT %
            </th>
            <th className="px-4 py-3 text-center font-semibold text-slate
              RISK
            </th>
            <th className="px-4 py-3 text-center font-semibold text-slate
              ACTION
            </th>
          </tr>
```

```
</thead>
<tbody>
  {filtered.map((stock, idx) => (
    <tr
      key={stock.id}
      className={`border-b border-slate-700/30 hover:bg-slate-700
        idx % 2 === 0 ? 'bg-slate-800/20' : 'bg-slate-900/20'
      }`}
    >
      <td className="px-4 py-3">
        <div className="flex items-center gap-2">
          <span className="font-bold text-white">{stock.symbol}</
          <span className="text-xs text-slate-500">{stock.sector}
        </div>
      </td>
      <td className="px-4 py-3 text-center">
        <span className="font-bold text-emerald-400">{stock.score
      </td>
      <td className="px-4 py-3 text-center">
        <span
          className={`text-xs font-semibold px-2 py-1 rounded-ful
            stock.signal
          )}`}
        >
          {stock.signal}
        </span>
      </td>
      <td className="px-4 py-3 text-center text-white font-semibo
        {stock.fromPivot.toFixed(1)}%
      </td>
      <td className="px-4 py-3 text-center font-semibold text-whi
        {stock.rsRank}
      </td>
      <td className="px-4 py-3 text-center text-white">
        {stock.baseQuality}
      </td>
      <td className="px-4 py-3 text-center">
        {stock.vcp ? (
          <CheckCircle className="w-4 h-4 text-green-500 mx-auto"
        ) : (
          <div className="w-4 h-4 border border-slate-600 mx-auto
        )}
      </td>
      <td className="px-4 py-3 text-center font-semibold text-eme
        {(stock.breakoutProbability * 100).toFixed(0)}%
      </td>
      <td className={`px-4 py-3 text-center font-semibold ${getRi
```

```
                  {stock.risk}
                </td>
                <td className="px-4 py-3 text-center">
                  <div className="flex items-center justify-center gap-1">
                    <button
                      onClick={() => setSelectedStock(stock)}
                      className="p-1.5 hover:bg-slate-600/50 rounded transi
                      title="View Chart"
                    >
                      <BarChart3 className="w-4 h-4 text-blue-400" />
                    </button>
                    <button
                      onClick={() => createAlert(stock, 'breakout')}
                      className="p-1.5 hover:bg-slate-600/50 rounded transi
                      title="Set Alert"
                    >
                      <Bell className="w-4 h-4 text-yellow-400" />
                    </button>
                    <button
                      onClick={() => addToWatchlist(stock)}
                      className="p-1.5 hover:bg-slate-600/50 rounded transi
                      title="Add to Watchlist"
                    >
                      <Eye className="w-4 h-4 text-emerald-400" />
                    </button>
                  </div>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  ) : (
    <div className="bg-slate-800/40 backdrop-blur border border-slate-700/5
      <AlertCircle className="w-12 h-12 text-slate-500 mx-auto mb-4" />
      <p className="text-slate-400">No stocks match your filters.</p>
    </div>
  )}
</div>

{/* Stock Details Modal */}
{selectedStock && (
  <div
    className="fixed inset-0 bg-black/70 backdrop-blur-sm z-50 flex items-c
    onClick={() => setSelectedStock(null)}
  >
```

```
<div
  className="bg-slate-900 border border-slate-700 rounded-xl max-w-4xl
  onClick={(e) => e.stopPropagation()}
>
  {/* Modal Header */}
  <div className="sticky top-0 bg-gradient-to-r from-slate-900 to-slate
    <div className="flex items-center gap-3">
      <div className="p-2 bg-gradient-to-br from-emerald-500 to-teal-60
        <TrendingUp className="w-5 h-5 text-white" />
      </div>
      <div>
        <h2 className="text-2xl font-bold text-white">{selectedStock.sy
        <p className="text-xs text-slate-400">
          {selectedStock.sector} • {selectedStock.marketCap} Market Cap
        </p>
      </div>
    </div>
    <button
      onClick={() => setSelectedStock(null)}
      className="text-slate-400 hover:text-white transition-colors"
    >
      <X className="w-6 h-6" />
    </button>
  </div>

  <div className="p-6 max-h-[calc(100vh-200px)] overflow-y-auto">
    {/* TradingView Chart Placeholder */}
    <div className="bg-slate-800/50 border border-slate-700 rounded-xl
      <div className="flex items-center justify-between mb-4">
        <h3 className="font-bold text-white flex items-center gap-2">
          <BarChart3 className="w-5 h-5 text-emerald-400" />
          Price Chart
        </h3>
        <button
          onClick={() =>
            copyToClipboard(`NSE:${selectedStock.symbol}`)
          }
          className="flex items-center gap-1 px-2 py-1 text-xs bg-slate
        >
          <Copy className="w-3 h-3" />
          Copy Symbol
        </button>
      </div>
      <div className="bg-slate-900/50 rounded-lg h-80 flex flex-col ite
        <div className="text-center">
          <Zap className="w-12 h-12 text-slate-500 mx-auto mb-2" />
          <p className="text-slate-400 font-semibold">TradingView Chart
```

```
          <p className="text-xs text-slate-500 mt-2 max-w-xs">
            Embed TradingView widget using your API key for {selectedSt
          </p>
          <button
            onClick={() =>
              copyToClipboard(getTradingViewWidget(selectedStock.symbol
            }
            className="mt-3 px-3 py-1 bg-emerald-600/20 hover:bg-emeral
          >
            Copy Widget Code
          </button>
        </div>
      </div>
    </div>

    {/* Price Levels */}
    <div className="grid grid-cols-2 md:grid-cols-5 gap-4 mb-6">
      <div className="bg-slate-800/50 border border-slate-700 rounded-l
        <p className="text-xs text-slate-400 mb-1">CURRENT</p>
        <p className="text-xl font-bold text-white">
          ₹{selectedStock.currentPrice.toFixed(2)}
        </p>
        <p className="text-xs text-slate-500 mt-1">
          {((selectedStock.currentPrice - selectedStock.pivot) / select
        </p>
      </div>
      <div className="bg-slate-800/50 border border-slate-700 rounded-l
        <p className="text-xs text-slate-400 mb-1">PIVOT</p>
        <p className="text-xl font-bold text-emerald-400">
          ₹{selectedStock.pivot.toFixed(2)}
        </p>
        <p className="text-xs text-emerald-500 mt-1">Breakout Level</p>
      </div>
      <div className="bg-slate-800/50 border border-slate-700 rounded-l
        <p className="text-xs text-slate-400 mb-1">RESISTANCE</p>
        <p className="text-xl font-bold text-red-400">
          ₹{selectedStock.resistance1.toFixed(2)}
        </p>
        <p className="text-xs text-slate-500 mt-1">
          R2: ₹{selectedStock.resistance2.toFixed(2)}
        </p>
      </div>
      <div className="bg-slate-800/50 border border-slate-700 rounded-l
        <p className="text-xs text-slate-400 mb-1">SUPPORT</p>
        <p className="text-xl font-bold text-blue-400">
          ₹{selectedStock.support1.toFixed(2)}
        </p>
```

```
            <p className="text-xs text-slate-500 mt-1">
              S2: ₹{selectedStock.support2.toFixed(2)}
            </p>
          </div>
          <div className="bg-slate-800/50 border border-slate-700 rounded-l
            <p className="text-xs text-slate-400 mb-1">TARGET</p>
            <p className="text-xl font-bold text-cyan-400">
              ₹{(selectedStock.resistance2 + (selectedStock.resistance2 - s
            </p>
            <p className="text-xs text-slate-500 mt-1">Projected</p>
          </div>
        </div>

        {/* Technical Analysis */}
        <div className="grid grid-cols-2 md:grid-cols-4 gap-4 mb-6">
          <div className="bg-slate-800/50 border border-slate-700 rounded-l
            <p className="text-xs text-slate-400 mb-2">RSI</p>
            <div className="flex items-end gap-2">
              <p className="text-2xl font-bold text-white">
                {selectedStock.rsi.toFixed(0)}
              </p>
              <div
                className="h-8 bg-gradient-to-t from-emerald-500 to-emerald
                style={{
                  width: `${(selectedStock.rsi / 100) * 40}px`,
                }}
              />
            </div>
          </div>
          <div className="bg-slate-800/50 border border-slate-700 rounded-l
            <p className="text-xs text-slate-400 mb-2">ADX</p>
            <div className="flex items-end gap-2">
              <p className="text-2xl font-bold text-white">
                {selectedStock.adx.toFixed(0)}
              </p>
              <div
                className="h-8 bg-gradient-to-t from-blue-500 to-blue-500/5
                style={{
                  width: `${(selectedStock.adx / 100) * 40}px`,
                }}
              />
            </div>
          </div>
          <div className="bg-slate-800/50 border border-slate-700 rounded-l
            <p className="text-xs text-slate-400 mb-1">TREND</p>
            <p
              className={`text-lg font-bold ${getTrendColor(
```

```
          selectedStock.trend
        )}`}
      >
        {selectedStock.trend}
      </p>
    </div>
    <div className="bg-slate-800/50 border border-slate-700 rounded-l
      <p className="text-xs text-slate-400 mb-1">STAGE</p>
      <p className="text-lg font-bold text-cyan-400">S{selectedStock.
    </div>
  </div>

  {/* Analysis Summary */}
  <div className="bg-slate-800/50 border border-slate-700 rounded-lg
    <h3 className="font-bold text-white mb-3 flex items-center gap-2"
      <Target className="w-5 h-5 text-emerald-400" />
      Analysis Summary
    </h3>
    <div className="grid grid-cols-2 md:grid-cols-3 gap-4 text-sm">
      <div>
        <p className="text-slate-400 text-xs mb-1">Base Quality</p>
        <div className="w-full bg-slate-700 rounded-full h-2">
          <div
            className="bg-gradient-to-r from-emerald-500 to-teal-500
            style={{
              width: `${selectedStock.baseQuality}%`,
            }}
          />
        </div>
        <p className="text-white font-semibold mt-1">
          {selectedStock.baseQuality}/100
        </p>
      </div>
      <div>
        <p className="text-slate-400 text-xs mb-1">Volume Quality</p>
        <div className="w-full bg-slate-700 rounded-full h-2">
          <div
            className="bg-gradient-to-r from-blue-500 to-cyan-500 h-2
            style={{
              width: `${selectedStock.volQuality}%`,
            }}
          />
        </div>
        <p className="text-white font-semibold mt-1">
          {selectedStock.volQuality}/100
        </p>
      </div>
```

```
    <div>
      <p className="text-slate-400 text-xs mb-1">RS Rank</p>
      <div className="w-full bg-slate-700 rounded-full h-2">
        <div
          className="bg-gradient-to-r from-purple-500 to-pink-500 h
          style={{
            width: `${selectedStock.rsRank}%`,
          }}
        />
      </div>
      <p className="text-white font-semibold mt-1">
        {selectedStock.rsRank}/100
      </p>
    </div>
    <div>
      <p className="text-slate-400 text-xs mb-1">VCP Pattern</p>
      <p className="text-lg font-bold">
        {selectedStock.vcp ? (
          <span className="text-green-400">✓ YES</span>
        ) : (
          <span className="text-slate-400">✗ NO</span>
        )}
      </p>
    </div>
    <div>
      <p className="text-slate-400 text-xs mb-1">Volume Dry-Up</p>
      <p className="text-lg font-bold">
        {selectedStock.volDry ? (
          <span className="text-green-400">✓ YES</span>
        ) : (
          <span className="text-slate-400">✗ NO</span>
        )}
      </p>
    </div>
    <div>
      <p className="text-slate-400 text-xs mb-1">Breakout Probabili
      <p className="text-lg font-bold text-emerald-400">
        {(selectedStock.breakoutProbability * 100).toFixed(0)}%
      </p>
    </div>
  </div>
</div>

{/* Action Buttons */}
<div className="flex gap-2 mb-4">
  <button
    onClick={() => {
```

```
                createAlert(selectedStock, 'breakout');
                addAlert(`Price alert set for ${selectedStock.symbol} at ₹${s
              }}
              className="flex-1 flex items-center justify-center gap-2 px-4 p
            >
              <Bell className="w-4 h-4" />
              Set Price Alert
            </button>
            <button
              onClick={() => {
                addToWatchlist(selectedStock);
                addAlert(`${selectedStock.symbol} added to watchlist`, 'succe
              }}
              className="flex-1 flex items-center justify-center gap-2 px-4 p
            >
              <Eye className="w-4 h-4" />
              Add to Watchlist
            </button>
            <button
              onClick={() =>
                copyToClipboard(
                  `${selectedStock.symbol} - Pivot: ₹${selectedStock.pivot},
                )
              }
              className="flex-1 flex items-center justify-center gap-2 px-4 p
            >
              <Copy className="w-4 h-4" />
              Copy Levels
            </button>
          </div>

          {/* Disclaimer */}
          <div className="p-4 bg-amber-500/10 border border-amber-500/30 roun
            <p className="text-xs text-amber-600 font-semibold">⚠ DISCLAIMER<
            <p className="text-xs text-amber-700 mt-1">
              This is a screening and analysis tool only. Not investment or t
              does not guarantee future results. Always conduct your own rese
              consult with a financial advisor before trading.
            </p>
          </div>
        </div>
      </div>
    )}
  </div>
);
}
```